2D to 3D Reconstruction — Deep Technical Presentation Notes

OVERVIEW OF THE FULL PIPELINE

```
Input Image (2D JPG/PNG)
■
▼
[Step 1] Image Loading & Color Space Conversion
■
▼
[Step 2] Image Processing
- Grayscale Conversion (LAB L-channel)
- Gaussian Blur (Noise Reduction)
- Canny Edge Detection
- Output: gray_processed.jpg (Guide Image)
■
▼
[Step 3] Depth Estimation (DepthAnything V2)
- Vision Transformer Inference
- Guided Filter Refinement (using grayscale guide)
- Normalization
- Output: depth_map.npy
■
▼
[Step 4] Point Cloud Generation
- Pinhole Camera Back-Projection
- 3 outputs: RGB, Grayscale, Heatmap (.ply files)
- Statistical Outlier Removal
■
▼
[Step 5] Mesh Generation
- Grid Meshing (Height-Map Tessellation)
- Vertex Color Assignment
- GLB Export via Trimesh
■
▼
[Step 6] Frontend (Gradio Web App)
- Interactive 3D Viewer (Model3D)
- Double-sided rendering
- White theme UI
```

STEP 1: IMAGE LOADING & COLOR SPACE CONVERSION

Why Color Space Matters

A digital image is stored as a 3D array of shape (Height, Width, 3).

The "3" represents three color channels. The order of those channels is NOT universal.

BGR (OpenCV default):

- Channel 0 = Blue
- Channel 1 = Green
- Channel 2 = Red

RGB (Standard for AI/Display):

- Channel 0 = Red

- Channel 1 = Green
- Channel 2 = Blue

If you feed a BGR image to a model expecting RGB, the red and blue channels are swapped.

A red apple would appear blue to the model. This causes completely wrong depth predictions.

Conversion

```
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
pil_image = Image.fromarray(img_rgb)
```

Why PIL Image?

HuggingFace `pipeline()` expects a PIL Image object, not a NumPy array.

PIL (Python Imaging Library) is the standard Python image format.

The conversion is lossless — no pixel data changes.

STEP 2: IMAGE PROCESSING (script: 02_image_processing.py)

2a. Grayscale Conversion — Why and How

The Problem with Simple RGB Grayscale

The naive approach is to average the three channels:

```
Gray = (R + G + B) / 3
```

This is mathematically simple but **perceptually wrong**.

Human eyes are NOT equally sensitive to all colors:

- We are most sensitive to **Green** (~59% contribution).
- Moderately sensitive to **Red** (~30% contribution).
- Least sensitive to **Blue** (~11% contribution).

If you average equally, a bright blue object and a dim green object might produce the same gray value, even though they look very different in brightness to a human.

The Luminance Formula (ITU-R BT.601 Standard)

```
Y = 0.299*R + 0.587*G + 0.114*B
```

This is the industry-standard formula used in TV broadcasting, JPEG compression, and professional image processing. It weights channels according to human visual perception.

The LAB Color Space (What We Actually Use)

We go one step further and use the **CIE LAB color space**:

```
lab = cv2.cvtColor(img, cv2.COLOR_RGB2LAB)
L, A, B = cv2.split(lab)
```

LAB has three channels:

- **L** = Lightness (0=black, 100=white) — This is what we use.
- **A** = Green-Red axis (color information).

- **B** = Blue-Yellow axis (color information).

Why LAB over simple grayscale?

1. LAB is **perceptually uniform** — equal numerical differences correspond to equal perceived differences in color/brightness.

2. The L channel **completely separates luminance from color** (chrominance). This means color changes (e.g., red object on green background) don't affect the L channel if both have the same brightness.

3. This gives us a more accurate "structural" representation of the scene, which is exactly what we need for depth refinement.

Practical effect: Object boundaries defined by brightness differences (not just color differences) are more accurately captured in the L channel than in simple grayscale.

2b. Gaussian Blur — Noise Reduction Before Edge Detection

What is Image Noise?

Digital cameras introduce random pixel-level variations called noise.

A pixel that should be value 120 might be recorded as 118 or 123.

These random variations are high-frequency signals — they change rapidly from pixel to pixel.

Why Blur Before Edge Detection?

Edge detection works by finding places where pixel values change rapidly (high gradient).

But noise also causes rapid pixel-to-pixel changes.

Without blurring, the edge detector would find thousands of fake "edges" from noise.

The Gaussian Kernel

A Gaussian blur replaces each pixel with a weighted average of its neighbourhood.

The weights follow a 2D Gaussian (bell curve) distribution:

```
w(x, y) = (1 / 2πσ²) * exp(-(x² + y²) / 2σ²)
```

Where:

- (x, y) = offset from center pixel
- σ = standard deviation (controls blur strength)
- The center pixel gets the highest weight
- Pixels further away get exponentially lower weights

For a 5×5 kernel (σ≈1), the weights look approximately like:

```
1 4 7 4 1
4 16 26 16 4
7 26 41 26 7 (normalized so all weights sum to 1)
4 16 26 16 4
1 4 7 4 1
```

Implementation

```
blurred = cv2.GaussianBlur(gray, (5, 5), 0)
```

- `(5, 5)` = kernel size (5×5 neighbourhood)

- `0` = let OpenCV calculate σ automatically from kernel size

Hyperparameter Testing: Kernel Size

Kernel	Effect	Problem
(3,3)	Minimal blur	Noise still present, fake edges
(5,5)	Good balance	**Chosen**
(7,7)	Strong blur	Real edges become soft/lost
(11,11)	Very strong	Object boundaries destroyed

2c. Canny Edge Detection — Finding Object Boundaries

Canny (1986) is still the gold standard for edge detection. It has 4 stages:

Stage 1: Gradient Computation (Sobel Operator)

Compute the rate of change (gradient) of pixel intensity in X and Y directions:

```
Gx = [[-1, 0, +1], Gy = [[-1, -2, -1],
[-2, 0, +2], [ 0, 0, 0],
[-1, 0, +1]] [+1, +2, +1]]
```

These are called **Sobel kernels**. Convolving with Gx finds vertical edges (horizontal changes).

Convolving with Gy finds horizontal edges (vertical changes).

Gradient magnitude: `G = sqrt(Gx² + Gy²)`

Gradient direction: `θ = arctan(Gy / Gx)`

Stage 2: Non-Maximum Suppression

Edges should be thin (1 pixel wide), but the gradient magnitude creates thick blurry bands.

For each pixel, check if it is a local maximum along the gradient direction.

If not, suppress it (set to 0). This thins the edges to 1 pixel.

Stage 3: Double Threshold (Hysteresis)

Two thresholds are applied:

- **High threshold (150):** Pixels above this are definitely edges ("strong edges").
- **Low threshold (50):** Pixels below this are definitely NOT edges (discarded).
- **Between 50-150:** "Weak edges" — kept only if connected to a strong edge.

This connectivity check is called **hysteresis thresholding** and is what makes Canny robust.

Stage 4: Edge Tracking by Connectivity

Walk along strong edges. Any weak edge pixel connected to a strong edge is promoted to a strong edge. Isolated weak edges are discarded.

Implementation

```
edges = cv2.Canny(blurred, 50, 150)
```

Hyperparameter Testing

Low / High	Effect
30 / 100	Too many edges, noise included
50 / 150	Clean edges, good detail — **Chosen**
80 / 200	Too few edges, misses fine detail
100 / 300	Only very strong edges kept

Role in Pipeline

The edge map is used as a **visual validation** tool — it confirms that the image has clear structural boundaries that the depth model can use. It is also saved as a diagnostic output.

STEP 3: DEPTH ESTIMATION (script: 03_depth_estimation.py)

What is Monocular Depth Estimation?

Monocular = using a single camera (one image).

Humans can estimate depth from a single eye using **monocular cues**:

- Objects that are larger appear closer.
- Objects higher in the frame appear further.
- Overlapping objects — the one in front is closer.
- Texture gradient — fine texture = far, coarse texture = close.
- Atmospheric haze — distant objects appear lighter/bluer.

DepthAnything V2 learns all these cues from millions of training images.

The Model: DepthAnything V2 (Small)

Architecture: Vision Transformer (ViT) + DPT Decoder

Encoder: Vision Transformer (ViT)

Traditional CNNs process images locally (small kernels). ViT processes images globally.

1. The input image is split into fixed-size **patches** (e.g., 16×16 pixels each).
2. Each patch is flattened into a vector and linearly projected to an embedding.
3. A **positional encoding** is added (so the model knows where each patch is).
4. These patch embeddings are fed into a stack of **Transformer blocks**.

Each Transformer block contains:

- **Multi-Head Self-Attention (MHSA):** Every patch attends to every other patch.

- This allows the model to relate distant parts of the image (e.g., sky patches inform ground depth).
- Formula: `Attention(Q, K, V) = softmax(QK^T / sqrt(d_k)) * V`
- Where Q (Query), K (Key), V (Value) are linear projections of the patch embeddings.

- **Feed-Forward Network (FFN):** Two linear layers with GELU activation.
- **Layer Normalization** and **Residual Connections** for training stability.

Decoder: Dense Prediction Transformer (DPT)

The ViT encoder produces feature maps at multiple scales.

DPT reassembles these into a full-resolution depth map:

1. Extract features from multiple ViT layers (not just the last one).

2. Upsample and fuse features at different resolutions (like a U-Net).

3. Final convolutional head produces a single-channel depth map.

Training Data

- Pretrained on **1.5 million+ images** with pseudo-depth labels.
- Pseudo-labels generated by larger teacher models (knowledge distillation).
- Fine-tuned on real depth datasets (NYU Depth V2, KITTI, etc.).

Output: Disparity Map

The model outputs a **disparity map**, not a true metric depth map.

- **Disparity** is inversely proportional to depth: `disparity = 1 / depth`
- **Higher disparity value = closer to camera = brighter pixel**
- This is why we do NOT invert the values when using them as Z coordinates.

Normalization

```
depth_norm = (depth_raw - depth_min) / (depth_raw.max() - depth_raw.min())
```

This maps the arbitrary float values to [0, 1] range.

Without this, the values could be any scale (e.g., 0.001 to 0.003) making them useless as coordinates.

Guided Filter — The Key Innovation

The Problem: Soft Depth Edges

DepthAnything V2 is excellent at estimating depth values but produces **soft edges** at object boundaries.

Why? The ViT processes 16×16 pixel patches. At boundaries between objects, a patch may contain pixels from both objects. The model averages these, producing a gradual transition instead of a sharp boundary.

This means the depth map has "bleeding" — the depth of a foreground object bleeds into the background pixels near the boundary.

The Solution: Guided Filter (He et al., ECCV 2010)

The guided filter uses a **high-quality guide image** (our sharp grayscale/L channel) to sharpen the edges of a **low-quality input** (the soft depth map).

Core Idea: Assume the output q is a linear transform of the guide I within each local window:

```
q_i = a_k * I_i + b_k for all i in window ω_k
```

Where a_k and b_k are constant within window ω_k but vary across windows.

Solving for a_k and b_k (minimizing the difference between q and the input p):

```
a_k = (cov(I, p) in ω_k) / (var(I) in ω_k + ε)
b_k = mean(p) in ω_k - a_k * mean(I) in ω_k
```

Final output (average over all windows containing pixel i):

```
q_i = mean(a)_i * I_i + mean(b)_i
```

What Each Parameter Does

Radius r (window size):

- Controls the size of the local window ω_k.
- Small r (e.g., 4): Only very local structure is transferred. Fine edges preserved but may miss larger structures.
- Large r (e.g., 16): Larger structures smoothed. May over-smooth fine details.
- **Chosen: r = 8** — good balance between edge sharpness and smoothness.

Epsilon ε (regularization):

- Controls edge preservation vs smoothing.
- Small ε (e.g., 0.001): Very edge-sensitive. Sharp edges in guide strongly influence output. May cause ringing artifacts.
- Large ε (e.g., 0.1): More smoothing. Edges in guide have less influence. Output is smoother but less sharp.
- **Chosen: ε = 0.01** — preserves important depth boundaries without artifacts.

Hyperparameter Testing

r	ε	Result
4	0.001	Too sharp, ringing artifacts at edges
4	0.01	Sharp but some noise
8	**0.01**	**Best balance — Chosen**
8	0.1	Slightly over-smoothed
16	0.01	Over-smoothed, loses fine detail

Effect on Point Cloud

Without guided filter: Object boundaries in the point cloud are blurry — foreground and background points blend together.

With guided filter: Sharp, clean boundaries — each object has a distinct depth layer.

Pinhole Camera Model — Back-Projection Formula

The Pinhole Camera Model

A pinhole camera maps 3D world points to 2D image pixels using perspective projection:

```
u = fx * (X/Z) + cx
v = fy * (Y/Z) + cy
```

Where:

- (X, Y, Z) = 3D world coordinates
- (u, v) = 2D pixel coordinates
- fx, fy = focal lengths in pixels
- (cx, cy) = principal point (optical center, usually image center)

Back-Projection (2D → 3D)

We invert this to go from 2D pixel + depth → 3D point:

```
X = (u - cx) * Z / fx
Y = (v - cy) * Z / fy
Z = d (depth value)
```

Focal Length Approximation

We don't have real camera calibration data, so we approximate:

```
focal_length = image_width * focal_length_scale # focal_length_scale = 0.8
cx = image_width / 2
cy = image_height / 2
```

This assumes the camera has a field of view roughly matching the image dimensions.

The `focal_length_scale = 0.8` was tuned to produce a natural-looking point cloud.

Hyperparameter Testing: focal_length_scale

Scale	Effect
0.5	Wide angle — points spread too far apart
0.8	Natural perspective — **Chosen**
1.0	Slightly narrow — points compressed
1.5	Very narrow — flat, compressed depth

Multi-View Point Cloud Generation (3 Outputs)

From the same depth map, three complementary point clouds are generated.

This is called **multi-modal output** — same data, different representations.

1. RGB Point Cloud

- Each 3D point is colored with the original image's RGB value at that pixel.
- **Purpose:** Photorealistic visualization. Shows what the scene actually looks like in 3D.
- **Use case:** Visual validation, presentations, demos.

2. Grayscale Point Cloud

- Each point is colored with the L-channel (luminance) value.
- **Purpose:** Structure analysis without color bias.
- **Why useful:** Color can mislead structural analysis. A red object on a green background has high color contrast but may have similar brightness. The grayscale view shows pure structural information.
- **Use case:** Scientific analysis, architectural reconstruction, medical imaging.

3. Heatmap Point Cloud (INFERNO colormap)

- Each point is colored by its depth value using the INFERNO colormap.
- Close points = bright yellow/white. Far points = dark purple/black.
- **Purpose:** Depth visualization for debugging and education.

Why INFERNO over JET colormap?

JET (the classic rainbow colormap) has serious problems:

1. **Not perceptually uniform:** Equal numerical differences don't look equal visually. The green-to-cyan transition looks much larger than the blue-to-cyan transition.

2. **Not colorblind-friendly:** Red-green colorblind people cannot distinguish large portions of the JET colormap.

3. **Non-monotonic luminance:** The brightness goes up and down, creating false "edges" in visualizations.

INFERNO:

1. **Perceptually uniform:** Equal depth differences look equal visually.

2. **Colorblind-friendly:** Distinguishable by all types of color vision.

3. **Monotonically increasing luminance:** Brightness always increases with depth value — no false edges.

4. **Reference:** "A Better Default Colormap for Matplotlib" (Smith & van der Walt, SciPy 2015).

Statistical Outlier Removal

```
cl, ind = pcd.remove_statistical_outlier(nb_neighbors=50, std_ratio=2.0)
```

What are Outliers in Point Clouds?

"Flying pixels" — isolated points that appear far from the main surface.

These come from:

- Depth estimation errors at object boundaries.
- Reflective surfaces (mirrors, glass) where depth is undefined.
- Transparent objects.
- Occlusion boundaries where depth "bleeds."

How Statistical Outlier Removal Works

For each point p:

1. Find its k nearest neighbours (k = nb_neighbors = 50).

2. Compute the mean distance to those neighbours: `mean_dist(p)`.

3. Compute the global mean and standard deviation of all mean distances.

4. If `mean_dist(p) > global_mean + std_ratio * global_std`, mark as outlier.

Points marked as outliers are removed.

Parameter Tuning

nb_neighbors	std_ratio	Effect
20	2.0	Fast but misses some outliers
50	**2.0**	Good balance — **Chosen**
50	1.0	Aggressive — removes too many valid points
100	2.0	Slow, marginal improvement

Coordinate Transform

```
pcd.transform([[1, 0, 0, 0],
[0, -1, 0, 0],
[0, 0, -1, 0],
[0, 0, 0, 1]])
```

This is a 4×4 homogeneous transformation matrix.

It flips the Y and Z axes:

- Y → -Y: Image Y goes downward (row 0 = top). 3D Y goes upward. Flip to correct.
- Z → -Z: Depth goes into the screen. 3D Z goes out of the screen. Flip to correct.
- X stays the same (no mirroring).

Without this transform, the point cloud appears upside-down and inside-out in viewers.

STEP 4: POINT CLOUD VISUALIZATION (script: 04_visualize_point_cloud.py)

Open3D Visualizer

Open3D is a C++ library with Python bindings for 3D data processing and visualization.

It uses OpenGL for rendering — hardware-accelerated, interactive.

```
vis = o3d.visualization.Visualizer()
vis.create_window(window_name="Point Cloud", width=1280, height=720)
vis.add_geometry(pcd)
```

Initial Camera Setup

```
vis.get_view_control().set_front([0, 0, -1]) # Camera looks along -Z axis
vis.get_view_control().set_lookat([0, 0, 0]) # Looking at origin
vis.get_view_control().set_up([0, -1, 0]) # Y-up convention
vis.get_view_control().set_zoom(0.8) # Slight zoom out
```

Why these values?

- `set_front([0, 0, -1])`: Camera is placed along the +Z axis, looking toward -Z. This means we look at the front face of the point cloud directly.
- `set_up([0, -1, 0])`: In image coordinates, Y increases downward. We flip it so Y increases upward in the viewer (standard 3D convention).
- `set_zoom(0.8)`: Slightly zoomed out so the full point cloud is visible without clipping.

What This Step Proves

1. The depth map correctly encodes 3D structure — the point cloud looks like the real scene.
2. The guided filter produces sharper edges — object boundaries are clean.
3. The three views (RGB, Grayscale, Heatmap) demonstrate multi-modal output capability.
4. The pipeline is end-to-end functional from a 2D image to interactive 3D visualization.

STEP 5: MESH GENERATION

Why Move from Point Cloud to Mesh?

A point cloud is a set of disconnected dots. It has no surface — you can see through it.

A mesh is a set of connected triangles forming a continuous surface.

Advantages of Mesh over Point Cloud:

- Can be textured (image painted on surface).
- Renders faster (GPU-optimized triangle rasterization).
- Compatible with standard 3D formats (OBJ, GLB, FBX).
- Can be used in game engines, AR, VR, web viewers.
- File size is smaller for the same visual quality.

What Was Tried: Poisson Surface Reconstruction

Mathematical Foundation

Poisson reconstruction (Kazhdan et al., 2006) solves a partial differential equation (PDE).

Given a point cloud with surface normals, define an **indicator function** χ:

- $\chi = 1$ inside the surface
- $\chi = 0$ outside the surface

The gradient of χ at the surface equals the inward surface normals:

```
∇χ = V (where V is the vector field of surface normals)
```

This is a **Poisson equation**: $\nabla^2\chi = \nabla \cdot V$

Solving this PDE gives us χ everywhere in space.

We then extract the surface as the **isosurface** where $\chi = 0.5$ (the boundary between inside and outside).

Why It Fails for Single-View Images

1. **Missing normals:** Poisson requires surface normals at every point. We only have depth, not normals. Estimated normals from depth are noisy.

2. **Incomplete data:** We only see the front face. The back of the object has no points. Poisson tries to "close" the surface — it invents a back face by extrapolating, creating a smooth blob/bubble.

3. **Over-smoothing:** The PDE solution is inherently smooth. Sharp corners and fine details are lost.

4. **Artifacts at boundaries:** Where the point cloud ends (at the edges of the image), Poisson creates unnatural "walls" or "caps."

Result: For single-view reconstruction, Poisson produces a smooth, closed blob that looks nothing like the original object.

What Was Used Instead: Grid Meshing (Height-Map Tessellation)

Core Concept

Instead of trying to reconstruct a surface from scattered points, we directly convert the **depth map grid** into a mesh.

The depth map is already a regular 2D grid of values. We treat it as a **height map** — a terrain where depth = elevation.

Algorithm

For every 2×2 block of neighbouring pixels (i,j), (i,j+1), (i+1,j), (i+1,j+1):

```
(i,j)■■■■■■■(i,j+1)
■ ■ T1 ■
■ ■ ■
■ T2 ■ ■
(i+1,j)■(i+1,j+1)
```

Split into 2 triangles:

- **T1:** (i,j), (i,j+1), (i+1,j)
- **T2:** (i,j+1), (i+1,j+1), (i+1,j)

Each vertex has:

- X = normalized column index (0 to 1)
- Y = normalized row index (0 to 1), flipped
- Z = depth value at that pixel (0 to 0.5)

Implementation

```
factor = 4
wv = w // factor # Downsampled width
hv = h // factor # Downsampled height

xx, yy = np.meshgrid(np.linspace(0, 1, wv), np.linspace(0, 1, hv))

# Faces
r, c = np.meshgrid(np.arange(hv-1), np.arange(wv-1), indexing='ij')
v1 = (r * wv + c).flatten()
v2 = v1 + 1
v3 = v1 + wv
v4 = v1 + wv + 1

faces = np.vstack([
np.column_stack([v1, v2, v3]), # Triangle 1
np.column_stack([v2, v4, v3]), # Triangle 2
])
```

Downsample Factor

A 512×512 image has 512×512 = 262,144 pixels.

Without downsampling, the mesh would have ~500,000 triangles — too large for a web viewer.

Factor	Vertices	Triangles	File Size	Visual Quality
1	262,144	~524,000	~50MB	Maximum
2	65,536	~131,000	~12MB	Very High
4	**16,384**	**~32,000**	**~3MB**	**High — Chosen**
8	4,096	~8,000	~0.8MB	Medium

Factor=4 gives a good balance: visually detailed, fast to load in browser.

Why Grid Meshing is Better Than Poisson for This Use Case

Property	Grid Meshing	Poisson
Speed	Milliseconds (CPU)	Seconds to minutes
Sharpness	Pixel-perfect	Smoothed
Texture mapping	Trivial (grid = UV)	Complex (needs parameterization)
Single-view artifacts	None	Blob/bubble back face
File size	Controllable	Unpredictable
Requires normals	No	Yes

Vertex Colors vs UV Texture Mapping

UV Texture Mapping (What Was Tried First)

UV mapping assigns each vertex a 2D coordinate (u, v) in texture space.

The renderer looks up the color at (u, v) in the texture image.

Formula:

```
u = x_pixel / (W - 1)
v = 1 - (y_pixel / (H - 1)) # Flip V because image Y goes down, texture V goes up
```

Problem: The texture image is a separate file (`.jpg`). The `.obj` file references it by filename.

In a web browser, loading external files from local paths is blocked by security policies (CORS).

The texture would load in a desktop viewer but fail in the browser.

Vertex Colors (Final Solution)

Instead of a separate texture file, bake the color directly into each vertex:

```
img_small = cv2.resize(img_np, (wv, hv))
colors_rgb = img_small.reshape(-1, 3)
colors_rgba = np.column_stack([colors_rgb, np.full(len(colors_rgb), 255)])
```

Each vertex stores its own RGBA color (Red, Green, Blue, Alpha=255).

When exported to GLB, the colors are embedded in the binary file.

No external files needed — the GLB is completely self-contained.

Trade-off: Vertex colors are lower resolution than texture mapping (one color per vertex, not per pixel). But at downsample factor=4, the visual difference is negligible.

GLB Format — Why It's the Right Choice

GLB = GL Transmission Format Binary

It is the binary version of **glTF** (GL Transmission Format), developed by the Khronos Group (same group that maintains OpenGL and Vulkan).

Why GLB over OBJ:

- OBJ is a text format — large file size, slow to parse.
- OBJ requires separate `.mtl` (material) and texture files — multiple files to manage.
- GLB is binary — compact, fast to load.

- GLB is self-contained — geometry, materials, textures all in one file.
- GLB is the standard for web 3D (Three.js, Babylon.js, model-viewer all support it natively).
- GLB is the standard for AR/VR (Apple Reality Composer, Google ARCore, Meta Quest).

Double-Sided Rendering

By default, 3D renderers only draw the **front face** of each triangle.

The "front" is determined by the **winding order** of vertices:

- Counter-clockwise winding = front face (visible).
- Clockwise winding = back face (invisible).

When you rotate the model to look at the back, the back faces are invisible (white/transparent).

Fix: Set `doubleSided=True` in the GLB material:

```
mat = trimesh.visual.material.PBRMaterial(doubleSided=True)
mesh.visual.material = mat
```

This tells the renderer to draw both sides of every triangle.

The GLB standard supports this via the `material.doubleSided` flag.

STEP 6: FRONTEND (app.py)

Gradio Framework

Gradio is a Python library for building ML demos.

It automatically generates HTML/CSS/JavaScript from Python code.

It runs a local web server (Flask-based) and serves the UI in the browser.

Why Gradio over Flask/Django?

- Zero HTML/CSS/JS required — pure Python.
- Built-in components for images, 3D models, files.
- Automatic API endpoint generation.
- One-line public sharing (`share=True` creates a temporary public URL via Gradio's tunnel).

gr.Model3D Component

This component uses **Google's** `<model-viewer>` **web component** under the hood.

`<model-viewer>` is a WebGL-based 3D viewer that supports GLB/GLTF files natively.

Requires WebGL — a browser API for GPU-accelerated 3D rendering.

WebGL uses the GPU to render triangles at 60fps.

Without WebGL (e.g., if hardware acceleration is disabled), the viewer shows nothing.

Settings used:

```
gr.Model3D(
clear_color=[1.0, 1.0, 1.0, 1.0], # White background (RGBA)
interactive=True, # Enable drag-to-rotate
height=500, # Viewer height in pixels
)
```

Coordinate System Fixes

The 3D viewer has its own coordinate system conventions.

Several fixes were needed to make the model appear correctly:

1. **X-axis flip** (`* -1`): Makes the mesh face the camera by default (otherwise it faces away).
2. **Image pre-flip** (`cv2.flip(img, 1)`): Cancels the mirror effect caused by the X-axis flip.
3. **Y-axis flip** (`* -1`): Image Y goes downward, 3D Y goes upward.
4. **Z normalization** (0 to 0.5): Ensures depth is proportional to the image dimensions.

Full Pipeline in app.py

```
def reconstruct(pil_image):
# 1. Depth estimation
result = depth_pipe(pil_image)
depth_np = np.array(result["depth"])

# 2. Downsample
factor = 4
wv, hv = w // factor, h // factor

# 3. Grid
xx, yy = np.meshgrid(np.linspace(0,1,wv), np.linspace(0,1,hv))

# 4. Colors (pre-flipped to cancel X-axis mirror)
img_flipped = cv2.flip(img_np, 1)
img_small = cv2.resize(img_flipped, (wv, hv))
colors_rgba = ...

# 5. Depth
z = (depth - min) / (max - min) * 0.5

# 6. Vertices (X flipped to face camera)
verts = np.column_stack([(xx-0.5)*-1, (yy-0.5)*-1, z])

# 7. Faces (triangulation)
faces = ...

# 8. Trimesh + GLB export
mesh = trimesh.Trimesh(vertices=verts, faces=faces, vertex_colors=colors_rgba)
mesh.visual.material = trimesh.visual.material.PBRMaterial(doubleSided=True)
mesh.export("model.glb")

return "model.glb"
```

QUICK ANSWER SHEET (For Q&A)

	Answer
ack empty?	Monocular depth (2.5D). Cannot hallucinate unseen geometry. Generative models (TripoSR) needed for that — they require GPU.
eshing over Poisson?	Poisson creates blob artifacts for single-view images, requires normals, and is slow. Grid meshing is instant, pixel-accurate, and trivially
me?	Yes. Local pipeline runs in under 3 seconds on CPU.
GLB file?	Binary 3D format (GL Transmission Format). Self-contained — geometry + colors in one file. Standard for web/AR/VR.
nything V2?	State-of-the-art monocular depth. Vision Transformer architecture. Pretrained on 1.5M+ images. Runs on CPU.

	Answer
Guided Filter for?	Sharpens depth map edges using the grayscale image as a structural guide. Transfers sharp boundaries from the photo to the soft dept
NO colormap?	Perceptually uniform, colorblind-friendly, monotonically increasing luminance. JET has all the opposite problems.
cloud outputs?	Multi-modal validation: RGB (visual), Grayscale (structural), Heatmap (depth analysis). Each serves a different analytical purpose.
lor space?	Perceptually uniform. L channel separates luminance from color — gives better structural information than simple RGB grayscale.
arity?	Inverse of depth. DepthAnything outputs disparity: bright = close, dark = far.
ample by factor 4?	Reduces mesh from 500k triangles to 32k. Keeps GLB under 5MB for fast browser loading.
bleSided?	GLB material flag that makes both faces of every triangle visible. Prevents white/invisible back face when rotating.
image before sampling colors?	The X-axis geometry flip (to face camera) mirrors the mesh. Pre-flipping the image cancels this mirror so colors appear correctly.